

AgentSight: System-Level Observability for AI Agents Using eBPF

Yusheng Zheng
UC Santa Cruz
Santa Cruz, CA, USA
yzhen165@ucsc.edu

Tong Yu
eunomia-bpf Community
China
yt.xyxx@gmail.com

Yanpeng Hu
ShanghaiTech University
Shanghai, China
huyp@shanghaitech.edu.cn

Andi Quinn
UC Santa Cruz
Santa Cruz, CA, USA
aquinn1@ucsc.edu

Abstract

Modern software infrastructure increasingly relies on LLM agents for development and maintenance, such as Claude Code and Gemini-cli. However, these AI agents differ fundamentally from traditional deterministic software, posing a significant challenge to conventional monitoring and debugging. This creates a critical semantic gap: existing tools observe either an agent’s high-level intent (via LLM prompts) or its low-level actions (e.g., system calls), but cannot correlate these two views. This blindness makes it difficult to distinguish between benign operations, malicious attacks, and costly failures. We introduce AgentSight, an AgentOps observability framework that bridges this semantic gap using a hybrid approach. Our approach, *boundary tracing*, monitors agents from outside their application code at stable system interfaces using eBPF. AgentSight intercepts TLS-encrypted LLM traffic to extract semantic intent, monitors kernel events to observe system-wide effects, and causally correlates these two streams across process boundaries using a real-time engine and secondary LLM analysis. This instrumentation-free technique is framework-agnostic, resilient to rapid API changes, and incurs less than 3% performance overhead. Our evaluation shows AgentSight detects prompt injection attacks, identifies resource-wasting reasoning loops, and reveals hidden coordination bottlenecks in multi-agent systems. AgentSight is released as an open-source project at <https://github.com/eunomia-bpf/agentsight>.

1 Introduction

The role of machine learning in systems is undergoing a fundamental shift from optimizing well-defined tasks, such as database query planning, to a new paradigm of *agentic computing*. From a systems perspective, an AI agent couples a Large Language Model’s (LLM) reasoning with direct access to system tools, granting it agency to perform operations like spawning processes, modifying the filesystem, and executing commands. This technology is being rapidly integrated

into production environments, powering autonomous developer tools like Claude Code[4], Cursor Agent[5] and Gemini-CLI[24], which can independently handle complex software engineering and system maintenance tasks. In essence, we are deploying non-deterministic ML systems, creating an unprecedented class of challenges for system reliability, security, and verification.

This paradigm shift creates a critical semantic gap: the chasm between an agent’s high-level *intent* and its low-level *system actions*. Unlike traditional programs with predictable execution paths, agents use LLMs and autonomous tools to dynamically generate code and spawn arbitrary subprocesses. This makes it hard for existing observability tools to distinguish benign operations from catastrophic failures. Consider an agent tasked with code refactoring that, due to a malicious prompt it reads from external url in the search result when search for API documents, instead injects a backdoor (indirect prompt injection)[30]. An application-level monitor might see a successful "execute script" tool call, while a system monitor sees a bash process writing to a file. Neither can bridge the gap to understand that a benign intention has been twisted into a malicious action, rendering them effectively blind.

Current approaches are trapped on one side of this semantic gap. *Application-level instrumentation*, found in frameworks like LangChain [8] and AutoGen [29], captures an agent’s reasoning and tool selection. While these tools see the *intent*, they are brittle, require constant API updates, and are easily bypassed: a single shell command escapes their view, breaking the chain of visibility under a flawed trust model. Conversely, *generic system-level monitoring* sees the *actions*, tracking every system call and file access. However, it lacks all semantic context. To such a tool, an agent writing a data analysis script is indistinguishable from a compromised agent writing a malicious payload. Without understanding the preceding LLM instructions, the *why* behind the *what*, its stream of low-level events is meaningless noise.

We propose boundary tracing as a novel observability method designed specifically to bridge this semantic gap.

AgentSight: 使用 eBPF 实现 AI 智能体的系统级可观测性

郑宇声 加州大学圣
克鲁兹分校 美国加利福
尼亚州圣克鲁兹
yzhen165@ucsc.edu

于通
eunomia-bpf社区 中国
yt.xyxx@gmail.com

胡艳鹏 上海科技大学 中国
上海
huyp@shanghaitech.edu.cn

安迪·奎恩 加州大
学圣克鲁兹分校 美国
加利福尼亚州圣克鲁兹
aquinn1@ucsc.edu

摘要

现代软件基础设施越来越依赖大语言模型智能体进行开发和运维，例如Claude Code和Gemini-CLI。然而，这些AI智能体在本质上与传统确定性软件存在根本差异，给常规监控和调试带来了重大挑战。由此产生了一个关键的语义鸿沟：现有工具只能观察智能体的高级意图（通过LLM提示）或其低级操作（例如系统调用），但无法将这两种视图关联起来。这种盲区使得难以区分良性操作、恶意攻击和高成本故障。我们推出了AgentSight，这是一个AgentOps可观测性框架，采用混合方法弥合这一语义鸿沟。我们的方法——边界追踪——利用eBPF，在稳定的系统接口上从智能体应用代码外部对其进行监控。AgentSight拦截TLS加密的大语言模型流量以提取语义意图，监控内核事件以观察系统级影响，并使用实时引擎和二级LLM分析跨进程边界因果关系这两股数据流。这种无需插桩的技术与框架无关，能够抵御API的快速变化，并且性能开销低于3%。我们的评估表明，AgentSight能够检测提示注入攻击，识别浪费资源的推理循环，并揭示多智能体系统中的隐藏协调瓶颈。AgentSight作为开源项目发布于<https://github.com/eunomia-bpf/agentsight>。

1 引言

机器学习在系统中的作用正经历从优化良定义任务（如数据库查询规划）到智能体计算这一新范式的根本性转变。从系统角度来看，AI智能体将大语言模型（LLM）的推理能力与对系统工具的直接访问相结合，赋予其自主性，以执行诸如创建进程、修改文件系统以及运行命令等操作。该技术正被迅速整合

被部署到生产环境中，为 Claude Code[4]、Cursor Agent[5] 和 Gemini-CLI[24]，等自主开发者工具提供动力，这些工具能够独立处理复杂的软件工程与系统维护任务。实质上，我们是在部署非确定性机器学习系统，从而为系统可靠性、安全性和验证性带来前所未有的挑战。

这一范式转变催生了一个关键的语义鸿沟：即智能体的高层意图与其底层系统操作之间的巨大差距。与执行路径可预测的传统程序不同，智能体利用大语言模型和自主工具动态生成代码并派生任意子进程。这使得现有的可观测性工具难以将良性操作与灾难性故障区分开来。设想一个负责代码重构的智能体，在搜索API文档时，从搜索结果中的外部URL读取了一个恶意提示，从而植入了一个后门（间接提示注入）[30]。应用程序级监控器可能看到一个成功的“执行脚本”工具调用，而系统监控器则看到一个bash进程正在向文件写入数据。两者均无法弥合这一鸿沟，因而无法理解良性意图已被扭曲为恶意行为，从而实际上处于“盲目”状态。

当前方法被困在这一语义鸿沟的一侧。应用层仪表化，如LangChain [8] 和AutoGen [29]，等框架中所采用的方法，能够捕捉智能体的推理和工具选择。虽然这些工具能够识别意图，但它们十分脆弱，需要持续更新API，且极易被绕过：一条Shell命令即可逃出其视野，在存在缺陷的信任模型下打破可见性链。相反，通用的系统级监控能够看到行为，追踪每一个系统调用和文件访问。然而，它完全缺乏语义上下文。对于这样的工具而言，一个正在编写数据分析脚本的智能体，与一个已遭入侵、正在编写恶意载荷的智能体是无法区分的。如果不理解先前的LLM指令——即行为背后的“为何”——那么由低级事件组成的数据流便毫无意义，仅是噪声。

我们提出边界追踪作为一种新颖的可观测性方法，专门用于弥合这一语义鸿沟。

Our key insight is that while agent internals and frameworks are volatile, the interfaces through which they interact with the world (the kernel for system operations and the network for communication) are stable and unavoidable. By monitoring from outside the application at these boundaries, we can capture an agent’s high-level intent and its low-level system effects. We present **AgentSight**, a system that realizes boundary tracing using eBPF to intercept TLS-encrypted LLM traffic for intent and monitor kernel events for effects. Its core is a novel, two-stage correlation process: a real-time engine links an LLM response to the system behavior it triggers, and a secondary "observer" LLM performs a deep semantic analysis on the resulting trace to infer risks and explain *why* a sequence of events is suspicious. This instrumentation-free, framework-agnostic technique incurs less than 3% overhead and effectively detects prompt injection attacks, resource-wasting reasoning loops, and multi-agent system bottlenecks.

In summary, our contributions are:

1. We introduce boundary tracing as a principled approach to AI agent observability that bridges the semantic gap by monitoring at stable system interfaces.
2. We present a novel engine that combines real-time, eBPF-based signal matching with LLM-based semantic analysis to provide deep, contextual understanding of agent behavior.
3. We demonstrate AgentSight’s effectiveness in detecting prompt injection attacks, reasoning loops, and multi-agent coordination failures with sub-3% overhead.

2 Background and Related Work

This section outlines LLM agent architecture, reviews existing observability work to highlight the semantic gap, and introduces eBPF as our foundational technology.

2.1 LLM Agent Architecture

The agentic systems described in the introduction are typically implemented using a common architecture. These systems consist of three core components: (1) an LLM backend for reasoning, (2) a tool execution framework for system interactions, and (3) a control loop that orchestrates prompts, tool calls, and state management. Popular frameworks such as LangChain [8], AutoGen [29], Cursor Agent[5], Gemini-CLI[24] and Claude Code[4] all implement variations of this model. This architecture is what enables agents to dynamically construct and execute complex plans (e.g., autonomously writing and running a script to analyze a dataset) based on high-level natural language objectives. Multi-agent LLM systems have also emerged for software development, collaborative task-solving, simulating social behaviors in virtual worlds, and other tasks [15, 28].

2.2 Observability for LLM Agent

Existing approaches are siloed on one side of the semantic gap. Intent-side observability, supported by industry tools like Langfuse, LangSmith, and Datadog [10, 16, 18, 19, 21] and is unifying by standards from the OpenTelemetry GenAI working group [7, 20] and academics conceptual taxonomies [11, 23] under the AgentOps concept, excels at tracing application-level events but is fundamentally blind to out-of-process system *actions*. Conversely, action-side observability with tools like Falco and Tracee [6, 27] offers comprehensive visibility into system calls but lacks the semantic context to understand an agent’s *intent*, failing to distinguish a benign task from a malicious one. A parallel line of research into reasoning-level and interpretability aims to make the agent’s internal thought processes more transparent by reconstructing cognitive traces [26] or enabling explanatory dialogues [17], but these work mainly focus on the llm itself, does not bridge the gap between the agent’s internal reasoning and its external, low-level effects on the system.

2.3 extended Berkeley Packet Filter (eBPF)

To bridge the semantic gap, our approach requires a technology that can safely and efficiently observe both network communication and kernel activity. eBPF (extended Berkeley Packet Filter) is a fundamental advancement in kernel programmability that provides precisely this capability [14]. Originally designed for packet filtering, eBPF has evolved into a general-purpose, in-kernel virtual machine that powers modern observability and security tools [12, 25], and not limited to Linux[22, 32]. For AI agent observability, eBPF is uniquely suited because it allows observation at the exact boundaries where agents interact with the world, enabling both TLS interception for semantic *intent* and syscall monitoring for system *actions* with minimal overhead. Critically, its kernel-enforced safety guarantees, including verified termination and memory safety, make it ideal for production environments and provide a stable foundation for our solution [9].

3 Design

The design of AgentSight is guided by a single imperative: to bridge the semantic gap between an agent’s intent and its actions. We achieve this through a novel observability method, boundary tracing, realized by a multi-signal correlation engine.

3.1 Challenges

The emergent and non-deterministic nature of AI agents fundamentally breaks traditional program observability paradigms, introducing two core challenges that original observability cannot address.

我们的核心洞见在于，尽管智能体内部和框架是易变的，但它们与世界交互的接口（用于系统操作的内核和用于通信的网络）是稳定且不可避免的。通过在教育外部对这些边界进行监控，我们可以捕捉智能体的高层意图及其底层系统影响。我们提出AgentSight，一个利用eBPF实现边界追踪的系统：它拦截TLS加密的大语言模型流量以获取意图，并监控内核事件以获取影响。其核心是一种新颖的两阶段关联过程：实时引擎将LLM响应与其触发的系统行为相关联，随后一个辅助的“观察者”LLM对生成的追踪记录进行深入语义分析，以推断风险并解释为何一系列事件可疑。这种无需插桩、框架无关的技术开销低于3%，并能有效检测提示词注入攻击、浪费资源的推理循环以及多智能体系统瓶颈。

总之，我们的贡献如下：

1. 我们引入了边界追踪作为AI智能体可观测性的原理性方法，通过在稳定的系统接口处进行监控来弥合语义鸿沟。
2. 我们提出了一种新型引擎，将基于eBPF的实时信号匹配与基于LLM的语义分析相结合，从而提供对智能体行为的深入、上下文感知的理解。
3. 我们展示了AgentSight在检测提示词注入攻击、推理循环和多智能体协调故障方面的有效性，其开销低于3%。

2 背景与相关工作

本节概述了大语言模型智能体架构，回顾了现有的可观测性相关研究以凸显语义鸿沟，并介绍了eBPF作为我们的基础技术。

2.1 大语言模型智能体架构

引言中描述的智能体系统通常采用一种通用架构来实现。这些系统由三个核心组件组成：（1）用于推理的大语言模型后端，（2）用于系统交互的工具执行框架，以及（3）协调提示、工具调用和状态管理的控制循环。流行的框架，如 LangChain [8].AutoGen [29], Cursor Agent[5], Gemini-CLI[24] 和 Claude Code[4]，均实现了该模型的变体。正是这种架构使得智能体能够基于高层级的自然语言目标，动态地构建并执行复杂的计划（例如，自主编写并运行一个脚本来分析数据集）。多智能体大语言模型系统也已出现在软件开发、协作任务求解、虚拟世界中的社会行为模拟以及其他任务中 [15, 28]。

2.2 LLM智能体的可观测性

现有方法孤立地存在于语义鸿沟的一侧。意图侧可观测性在Langfuse、LangSmith和Datadog [10, 16, 18, 19, 21]等行业工具的支持下，并由OpenTelemetry GenAI工作组 [7, 20] 制定的标准以及AgentOps概念下的学术概念分类法 [11, 23] 进行统一，擅长跟踪应用级事件，但根本上对进程外的系统操作视而不见。相反，采用Falco和Tracee [6, 27]等工具的操作侧可观测性提供了对系统调用的全面可见性，但缺乏理解智能体意图的语义上下文，无法区分良性任务与恶意任务。另一并行研究路径聚焦于推理级别和可解释性，旨在通过重建认知痕迹 [26] 或启用解释性对话 [17], 来提高智能体内部思维过程的透明度，但这些工作主要关注LLM本身，未能弥合智能体内部推理与其对系统产生的外部底层影响之间的鸿沟。

2.3 扩展Berkeley包过滤器（eBPF）

为了弥合语义鸿沟，我们的方法需要一种能够安全且高效地观察网络通信和内核活动的技术。eBPF（扩展的伯克利包过滤器）是内核可编程性的一项根本性进步，提供了正是这种能力 [14]。eBPF 最初是为包过滤而设计的，现已演变为一种通用的内核虚拟机，为现代可观测性和安全工具提供支持 [12, 25],，并且不仅仅局限于Linux [22, 32]。对于 AI 智能体可观测性而言，eBPF 具有独特优势，因为它允许在智能体与世界交互的确切边界处进行观察，从而以最小的开销实现用于语义意图的 TLS 拦截以及用于系统操作的系统调用监控。关键的是，其由内核强制实施的安全保证，包括经过验证的终止和内存安全，使其非常适合生产环境，并为我们解决方案提供了稳定的基础 [9]。

3 设计

AgentSight的设计遵循一个核心原则：弥合智能体的意图与其行动之间的语义鸿沟。我们通过一种新颖的可观测性方法——边界追踪——来实现这一目标，该方法由一个多信号关联引擎实现。

3.1 挑战

AI智能体的涌现性和非确定性从根本上打破了传统程序可观测性范式，引入了原始可观测性无法解决的两大核心挑战。

Bridging the Semantic Gap Between Intent and Action

The first and most significant challenge is bridging the vast semantic gap between an agent’s high-level *intent* and its low-level system *actions*. Unlike conventional software, where intent is encoded in predictable source code, an agent’s intent is expressed in natural language and interpreted by an LLM, creating dynamic "source code" that is generated at runtime. Consequently, it is impossible for a static analyzer to determine what an agent will do. For example, the intent "find and fix the bug in the authentication module" is semantically rich but operationally ambiguous, potentially resulting in a complex sequence of actions like reading files (openat2), compiling code (execve -> gcc), and running tests (execve -> python). This creates a critical observability problem: how can a monitoring system verify that the cascade of system calls is a legitimate fulfillment of the natural language intent? To solve this, an observer must move beyond simple pattern matching to gain a semantic understanding of the agent’s goal, necessitating a new, llm based approach to interpret the correlated traces.

Isolating the Causal Signal from High-Volume System Noise

The second challenge stems from the agent’s autonomy to use any tool necessary to achieve its goal, leading to an unpredictable and high-volume stream of system events. An agent might spawn shells, download scripts, or invoke compilers-processes that are not known ahead of time. This makes it exceedingly difficult to distinguish the agent’s specific activity (the "signal") from the background noise of the operating system. Static, pre-configured filters, for instance, a rule to only monitor git commands, are inherently brittle and will fail the moment the agent uses curl and bash to achieve a similar outcome. Our design addresses this with an aggressive, dynamic in-kernel eBPF filter. By tracking process creation events (fork, execve), the filter builds a complete lineage tree of the agent’s activity and dynamically applies rules in the kernel to only pass events from the agent or its descendants to userspace. This approach ensures that the entire causal chain is captured efficiently at the source, dramatically reducing overhead and providing a clean, high-fidelity signal for correlation and analysis.

3.2 Boundary Tracing: A Principled Approach

Our key insight is that all agent interactions must traverse well-defined and stable system boundaries: the kernel for system operations and the network for external communications with LLM serving backends (Figure 1). By monitoring at these boundaries rather than within volatile agent code, we achieve comprehensive monitoring independent of implementation details. This approach enables Semantic Correlation, the ability to causally link high-level intentions with low-level system events. This is supported by two principles. First is Comprehensiveness, as kernel-level monitoring ensures no system action from process creation to file I/O goes unobserved, even across spawned subprocesses. Second

is Stability, since system call ABIs and network protocols evolve far more slowly than agent frameworks, providing a durable, future-proof solution. This paradigm shifts the trust model from assuming a cooperative agent to enforcing observation at tamper-proof boundaries.

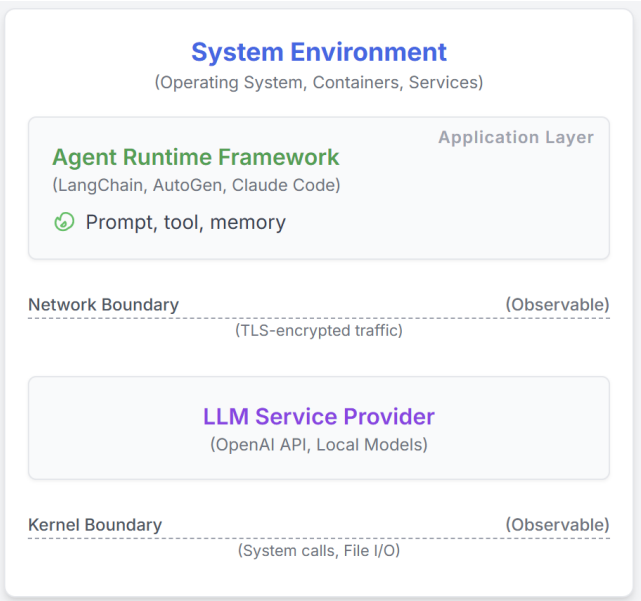


Figure 1. Agent Framework Overview

3.3 System Architecture: Observing the Boundaries

AgentSight’s architecture simultaneously taps into the two critical boundaries. As shown in Figure 2, we use eBPF to place non-intrusive probes that capture a decrypted Intent Stream (LLM prompts/responses) from userspace SSL functions and an Action Stream (syscalls, process events) from the kernel. A userspace correlation engine then processes and joins these streams into a unified, causally-linked trace.

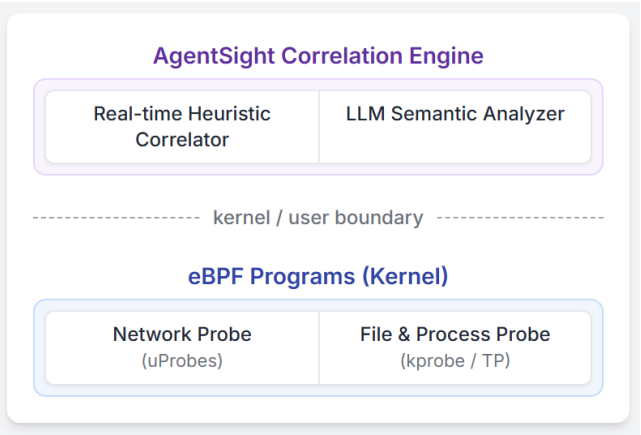


Figure 2. AgentSight System Architecture.

弥合意图与操作之间的语义鸿沟 首要且最具挑战性的任务是弥合智能体高层意图与其底层系统操作之间巨大的语义鸿沟。与传统软件不同，传统软件的意图以可预测的源代码形式编码，而智能体的意图以自然语言表达，并由 LLM 进行解释，从而生成在运行时动态产生的“源代码”。因此，静态分析器无法确定智能体将执行什么操作。例如，“找到并修复认证模块中的漏洞”这一意图在语义上丰富，但在操作上具有模糊性，可能导致一系列复杂的动作，如读取文件（openat2）、编译代码（execve -> gcc）以及运行测试（execve -> Python）。这造成了一个关键的观测性问题：监控系统如何验证系统调用的级联是自然语言意图的合法实现？为了解决这一问题，观察者必须超越简单的模式匹配，以获得对智能体目标的语义理解，这亟需一种基于 LLM 的新方法来解释关联跟踪。

将因果信号从海量系统噪声中分离出来 第二个挑战源于智能体的自主性：智能体为了达成目标，可以使用任何必要的工具，从而导致产生不可预测且高容量的系统事件流。智能体可能会创建 Shell、下载脚本或调用编译器——这些进程在事前是未知的。这使得将智能体的特定活动（即"信号"）与操作系统的背景噪声区分开来变得极其困难。例如，诸如仅监控 Git 命令这样的静态预配置 eBPF 过滤器本质上非常脆弱，一旦智能体使用 curl 和 bash 来达成类似的结果，这些过滤器就会失效。我们的设计通过一种积极的、动态的内核中 eBPF 过滤器来解决这一问题。通过跟踪进程创建事件（fork、execve），该过滤器可以构建出智能体活动的完整谱系树，并动态地在内核中应用规则，仅将来自智能体或其后代的事件传递到用户空间。这种方法确保整个因果链在源头处被高效地捕获，从而大幅降低开销，并为相关性和分析提供干净、高保真的信号。

3.2 边界追踪：一种有原则的方法

我们的核心洞察是：所有智能体交互都必须穿越明确定义且稳定的系统边界：即用于系统操作的内核，以及用于与 LLM 服务后端进行外部通信的网络（图1）。通过在边界处而非易变的智能体代码内部进行监控，我们实现了独立于实现细节的全面监控。这种方法使语义关联（Semantic Correlation）成为可能，即在高层意图与低层系统事件之间建立因果联系。这由两个原则支持。第一个是全面性，内核级监控确保从进程创建到文件 I/O 的所有系统行为均不被遗漏，即使跨由生成的子进程也是如此。第二个

这是稳定性，因为系统调用ABI和网络协议的演变远慢于智能体框架，从而提供了一种持久且面向未来的解决方案。这一范式将信任模型从假设合作型智能体转变为在防篡改边界处实施观察。

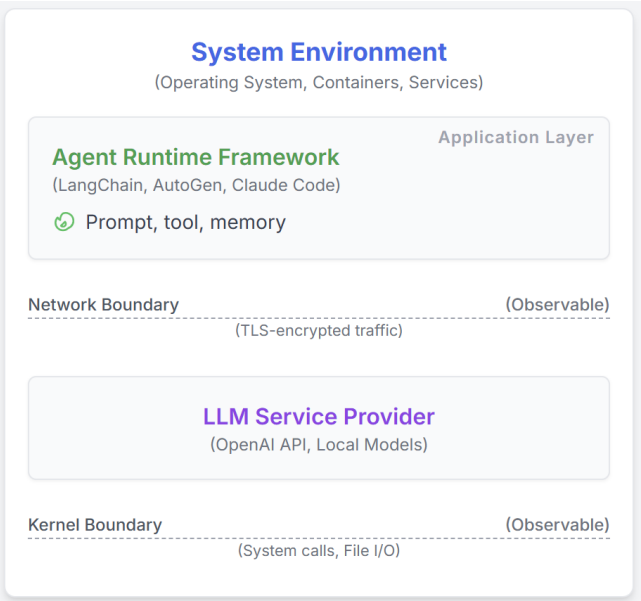


图1. 智能体框架概览

3.3 系统架构：观察边界

AgentSight 的架构同时接入两个关键边界。如图 2 所示，我们使用 eBPF 部署非侵入式探针，以捕获来自用户空间 SSL 函数的已解密意图流（LLM提示/响应），以及来自内核的动作流（系统调用、进程事件）。随后，用户态关联引擎对这些流进行处理和关联，形成一个统一的、因果关联跟踪。

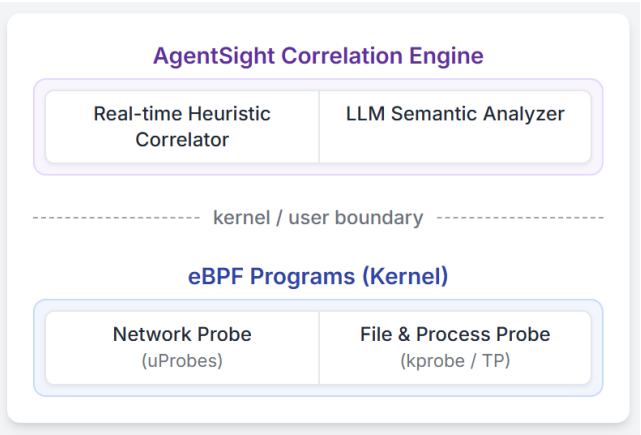


图2. AgentSight系统架构。

Several key components enable AgentSight to effectively bridge the semantic gap:

eBPF for Safe, Unified Probing: We chose eBPF for its production safety, high performance, and unified ability to access both userspace and kernel data streams. Our design intercepts decrypted data from the agent’s interaction with LLM serving backend, which is more efficient and manageable than network-level packet capture or proxy-based solutions.

Multi-Signal Causal Correlation Engine: The core of our design is a correlation strategy that establishes causality between intent and action. We designed a multi-signal engine that relies on three key mechanisms: Process Lineage, which builds a complete process tree by tracking fork and execve events to link actions in child processes back to the parent agent; Temporal Proximity, which associates actions that occur within a narrow time window immediately following an LLM response; and Argument Matching, which directly matches content from LLM responses, such as filenames, URLs, or commands, with the arguments of subsequent system calls. Together, these signals enable AgentSight to definitively establish causal relationships between high-level intentions and low-level system operations across process boundaries.

LLM-Powered Semantic Analysis: To move beyond brittle, rule-based detection, we designed the system to use a secondary LLM as a reasoning engine. By prompting a powerful model with the correlated event trace, we leverage its ability to understand semantic nuance, infer causality in complex scenarios, and summarize findings in natural language. This "AI to watch AI" approach allows AgentSight to detect threats that do not match predefined patterns.

4 Implementation

AgentSight is implemented as a userspace daemon (6000 lines of Rust/C) orchestrating eBPF programs, with a TypeScript frontend (3000 lines) for analysis. It is designed for high performance, processing raw kernel event streams into correlated, human-readable data.

4.1 Data Collection at the Boundaries

Our eBPF probes capture the raw intent and action streams from the system. To capture semantic intent, an eBPF program with uprobes attaches to SSL_read/SSL_write in crypto libraries like OpenSSL to intercept decrypted LLM communications. Our userspace daemon implements a stateful reassembly mechanism to handle streaming protocols such as Server-Sent Events (SSE). To capture system actions, a second eBPF program uses stable tracepoints like sched_process_exec to build a process tree and kprobes to dynamically monitor relevant syscalls such as openat2, connect, and execve. To manage the high volume of kernel events without data loss, aggressive in-kernel filtering is applied to

ensure only events from targeted agent processes are sent to userspace, minimizing overhead.

4.2 The Hybrid Correlation Engine

The Rust-based userspace daemon houses our two-stage correlation engine. The first stage consumes events from eBPF ring buffers and performs real-time heuristic linking. This streaming pipeline enriches raw events with context like mapping a file descriptor to a full path, maintains a stateful process tree, and applies the causal linking logic described in our design, using a 100-500ms window for temporal correlation. Once a coherent trace is constructed, the second stage formats it into a structured log for semantic analysis. This log is used to construct a detailed prompt for a secondary LLM, instructing it to act as a security analyst. The LLM’s natural language analysis and confidence score become the final output of our system. A key challenge at this stage is managing the latency and cost of LLM analysis, which our system mitigates through asynchronous processing and robust prompt engineering.

5 Evaluation

Our evaluation is guided by two research questions: First, what is the performance overhead of AgentSight in realistic workflows? Second, how effectively does it bridge the semantic gap to detect critical security threats and performance pathologies, while also revealing complex dynamics in multi-agent systems?

5.1 Performance Evaluation

Table 1. Overhead Introduced by AgentSight

Task	Baseline (s)	AgentSight (s)	Overhead
Understand Repo	127.98	132.33	3.4%
Code Writing	22.54	23.64	4.9%
Repo Compilation	92.40	92.72	0.4%

We evaluated AgentSight on a server (Ubuntu 22.04, Linux 6.14.0) using Claude Code 1.0.62[4] with claude 4 as the test agent. The benchmarks focused on three real-world developer workflows using a tutorial repo[13]: repository understanding with the /init command, code generation for bpfttrace scripts, and full repository compilation with parallel builds. Each experiment was run 3 times with and without AgentSight to measure runtime overhead. Table 1 quantifies the runtime overhead of AgentSight across three developer workflows, with an average 2.9% overhead.

5.2 Case Studies

We evaluated AgentSight’s effectiveness through case studies that demonstrate its ability to detect security threats, identify

AgentSight 能够有效弥合语义鸿沟的多个关键组件包括:

eBPF安全统一探测: 我们选择eBPF, 是因为其在生产环境中的安全性、高性能, 以及统一访问用户空间与内核数据流的能力。我们的设计通过截获智能体与LLM服务后端交互时解密后的数据, 这比网络层数据包捕获或基于代理的解决方案更高效、更易管理。

多信号因果关联引擎: 我们设计的核心是一种关联策略, 用于在意图与行为之间建立因果性。我们设计了一个多信号因果关联引擎, 依赖以下三个关键机制: 进程谱系 (Process Lineage), 通过跟踪fork和execve事件构建完整的进程树, 将子进程中的行为关联回父代理; 时间邻近性, 将紧随LLM响应之后、在极窄时间窗口内发生的行为相关联; 参数匹配 (Argument Matching), 直接将LLM响应中的内容 (如文件名、URL或命令) 与后续系统调用的参数进行匹配。这些信号共同使AgentSight能够在进程边界之间, 确定性地建立高层意图与底层系统操作之间的因果关系。

LLM驱动语义分析: 为了超越脆弱且基于规则的检测, 我们设计系统使用二级LLM作为推理引擎。通过向强大模型提供关联事件轨迹进行提示, 我们利用其理解语义细微差别、在复杂场景中推断因果性以及用自然语言总结发现的能力。这种“AI监视AI”的方法使AgentSight能够检测不符合预定义模式的威胁。

4 实现

AgentSight 以用户空间守护进程的形式实现 (6000 行 Rust/C 代码), 用于编排 eBPF 程序, 并配备 TypeScript 前端 (3000 行代码) 进行分析。其设计注重高性能, 能够将原始内核事件流处理为经过关联、可供人类阅读的数据。

4.1 边界处的数据采集

我们的 eBPF 探针捕获来自系统的原始意图和动作流。为了捕获语义意图, 一个带有 uprobes 的 eBPF 程序会附加到 OpenSSL 等加密库中的 SSL_read/SSL_write, 以拦截解密后的 LLM 通信。我们的用户空间守护进程实现了有状态的重组机制, 以处理服务器发送事件 (SSE) 等流式协议。为了捕获系统动作, 第二个 eBPF 程序使用稳定的跟踪点 (如 sched_process_exec) 来构建进程树, 并使用 kprobes 动态监控相关系统调用, 例如 openat2、connect 和 execve。为了在无数据丢失的情况下管理大量的内核事件, 会应用激进的内核内过滤,

确保仅将来自目标智能体进程的事件发送到用户空间, 从而最小化开销。

4.2 混合关联引擎

基于Rust的用户空间守护进程承载了我们的两级关联引擎。第一阶段从eBPF环形缓冲区消费事件, 并执行实时启发式链接。该流式管道通过映射文件描述符至完整路径等方式, 为原始事件丰富上下文信息, 维护有状态的进程树, 并应用设计中所述的因果链接逻辑, 采用100至500毫秒的时间窗口进行时间关联。一旦构建出连贯的跟踪, 第二阶段将其格式化为结构化日志以供语义分析。该日志用于为二级LLM构建详细的提示, 指示其充当安全分析师。LLM的自然语言分析和置信度分数构成了系统的最终输出。此阶段的一个关键挑战是管理LLM分析的延迟和成本, 我们的系统通过异步处理和健壮的提示工程来缓解这一问题。

5 评估

我们的评估由两个研究问题指导: 第一, AgentSight在真实工作流中的性能开销是多少? 第二, 它能否有效弥合语义鸿沟, 以检测关键安全威胁和性能病理, 同时揭示多智能体系统中的复杂动态?

5.1 性能评估

表1. AgentSight引入的开销

Task	基线 (秒)	AgentSight (s)	开销
理解代码仓库	127.98	132.33	3.4%
代码编写	22.54	23.64	4.9%
代码仓库编译	92.40	92.72	0.4%

我们在服务器上评估了 AgentSight (Ubuntu 22.04, Linux 6.14.0), 并使用 Claude Code 1.0.62[4] 配合 Claude 4 作为测试智能体。基准测试聚焦于使用教程代码仓库[13]的三个实际开发者工作流: 通过 /init 命令进行代码库理解、生成 bpfttrace 脚本的代码生成, 以及使用并行构建进行整个代码库编译。每个实验均运行了 3 次, 分别在开启和关闭 AgentSight 的情况下测量运行时开销。表 1 量化了 AgentSight 在三个开发者工作流中的运行时开销, 平均开销为 2.9%。

5.2 案例研究

我们通过案例研究评估了 AgentSight 的有效性, 这些案例研究展示了其检测安全威胁、识别

performance issues, and provide insights into complex multi-agent systems.

5.2.1 Case Study 1: Detecting Prompt Injection Attacks. We tested AgentSight’s ability to detect indirect prompt injection attacks[30]. In our test, a software development agent is instructed to clone and build a C project. The project’s README file directed the agent to a URL serving HTML with a hidden prompt. This prompt made the agent read and send /etc/passwd to a collection server, using a command cleverly disguised as a necessary step in the build process. AgentSight captured the full correlated attack chain: from the initial URL fetch to the final network exfiltration, with 521 events and merged into 37 events by Correlation Engine. The observer LLM analyzed this trace, returned a high-confidence attack score, and concluded that the agent’s actions were logically inconsistent with its stated goal. This result demonstrates how causally linking intent to system-level actions provides effective, context-aware threat detection.

5.2.2 Case Study 2: Reasoning Loop Detection. An agent attempting a complex task may enter an infinite loop due to a common tool usage error[31] . We implement a research agent using crewai[2] with gpt-4o-mini[1], it repeatedly called a web search tool with incorrect arguments, received an error, but then failed to correct its mistake, retrying the exact same failing command. AgentSight’s real-time monitors detect this anomalous resource consumption from a trace of API calls and passed it to the observer LLM. The LLM identified the root cause as a persistent tool error, noting the agent was caught in a "try-fail-re-reason" loop; it executed the same failing command, passed the identical error back to the reasoning LLM, and failed to learn from the tool’s output.

5.2.3 Case Study 3: Multi-Agent Coordination Monitoring. AgentSight monitored a team of 6 collaborating software development agents for our Github repo, using claude-code subagents[3], captured 3153 total events after Correlation Engine. For instance, frontend agent and test agent sometimes are blocked by sequential dependencies and numerous retry cycles caused by file locking contention during parallel development and testing tasks. The analysis demonstrated that while the agents developed some emergent coordination, separating the roles more clear could reduce total runtime and token cost. This reveals how boundary tracing uniquely captures multi-agent system dynamics that application-level monitoring cannot observe across process boundaries.

6 Conclusion

This paper introduced AgentSight to bridge the critical semantic gap between an AI agent’s intent and its system-level

actions using novel *boundary tracing* approach. By leveraging eBPF, the system monitors network and kernel events without instrumentation, causally linking LLM communications to their system-wide effects via a hybrid correlation engine. Our evaluation shows AgentSight effectively detects prompt injection attacks, reasoning loops, and multi-agent bottlenecks with under 3% performance overhead. This "AI to watch AI" provides a foundational methodology for the secure and reliable deployment of increasingly autonomous AI systems.

References

[1] 2024. GPT-4o Mini: Advancing Cost-Efficient Intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. Blog post announcing GPT-4o mini.

[2] 2025. crewAI: Framework for Orchestrating Role Playing Autonomous AI Agents. <https://github.com/crewAIInc/crewAI>. GitHub repository, accessed 30 Jul 2025.

[3] 2025. Subagents. <https://docs.anthropic.com/en/docs/claude-code/sub-agents>. Claude Code documentation page describing custom AI sub agents.

[4] Anthropic. 2025. Introducing Claude Code. <https://www.anthropic.com/news/claude-code>. Agentic coding tool announcement, Anthropic blog.

[5] Anysphere Inc. 2025. Cursor: The AI powered Code Editor. <https://cursor.com/>. AI assisted IDE with agent mode; latest release version 1.0 on June 4, 2025.

[6] The Falco Authors. 2023. Falco: Cloud Native Runtime Security. <https://falco.org/>

[7] Alexandr Bandurchin. [n. d.]. AI Agent Observability Explained: Key Concepts and Standards. <https://uptrace.dev/blog/ai-agent-observability>. Uptrace Blog, April 16, 2025.

[8] Harrison Chase. 2023. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>

[9] Linux Kernel Community. 2023. BPF Documentation - The Linux Kernel. <https://www.kernel.org/doc/html/latest/bpf/>

[10] Datadog Inc. [n. d.]. Monitor, troubleshoot, and improve AI agents with Datadog. <https://www.datadoghq.com/blog/monitor-ai-agents/>. Datadog Blog, 2023.

[11] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. *arXiv preprint arXiv:2411.05285* (2024).

[12] eBPF Community. 2023. eBPF Documentation. <https://ebpf.io/>

[13] eunomia-bpf. 2024. eBPF Developer Tutorial. <https://github.com/eunomia-bpf/bpf-developer-tutorial>. Accessed: 2025-01-29.

[14] Brendan Gregg. 2019. *BPF Performance Tools*. Addison-Wesley Professional.

[15] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680* (2024).

[16] Helicone. 2023. Helicone / LLM-Observability for Developers. <https://www.helicone.ai/>

[17] Been Kim, John Hewitt, Neel Nanda, Noah Fiedel, and Oyvind Tafjord. 2025. Because we have LLMs, we Can and Should Pursue Agentic Interpretability. *arXiv preprint arXiv:2506.12152* (2025).

[18] LangChain. 2023. Observability Quick Start - LangSmith - LangChain. <https://docs.smith.langchain.com/observability>

[19] Langfuse. 2024. Langfuse - LLM Observability & Application Tracing. <https://langfuse.com/>

[20] Guangya Liu and Sujay Solomon. [n. d.]. AI Agent Observability – Evolving Standards and Best Practices. <https://opentelemetry.io/blog/2025/ai-agent-observability/>. OpenTelemetry Blog, March 6, 2025.

性能问题，以及对复杂多智能体系统提供见解的能力。

5.2.1 案例研究 1：检测提示词注入攻击。我们测试了 AgentSight 检测间接提示注入攻击的能力[30]。在我们的测试中，一个软件开发智能体被指示克隆并构建一个 C 项目。该项目的 README 文件将该智能体引导至一个提供带有隐藏提示的 HTML 的 URL。该提示使该智能体读取 /etc/passwd 并发送到收集服务器，而所使用命令被巧妙地伪装成构建过程中的必要步骤。AgentSight 捕获了完整的关联攻击链：从最初的 URL 获取到最终的网络数据外传，共 521 个事件，并由关联引擎合并为 37 个事件。观察者 LLM 分析了此轨迹，返回了高置信度的攻击评分，并得出结论认为该智能体的行为与其既定目标在逻辑上不一致。此结果展示了如何将意图与系统级行动进行因果关联，从而提供有效的、上下文感知的威胁检测。

5.2.2 案例研究 2：推理循环检测。尝试执行复杂任务的智能体可能会由于常见的工具使用错误而进入无限循环[31]。我们使用 crewai[2]和 gpt-4o-mini[1], 实现了一个研究智能体；它反复以错误的参数调用一个网络搜索工具，收到错误后却未能纠正其错误，而是重试了完全相同的失败命令。AgentSight 的实时监控器从 API 调用轨迹中检测到这种异常的资源消耗，并将其传递给观察者 LLM。该 LLM 将根本原因识别为持续存在的工具错误，指出该智能体陷入了一种“尝试-失败-重新推理”循环；它执行了相同的失败命令，将完全相同的错误传回给推理 LLM，并且未能从工具的输出中吸取教训。

5.2.3 案例研究 3：多智能体协调监控。AgentSight 监控了为我们的 GitHub 仓库协作工作的 6 个软件开发智能体团队，使用 claude-code 子智能体[3], 在相关性引擎处理后共捕获了 3153 个事件。例如，前端智能体和测试智能体有时会因并行开发和测试任务中的文件锁定争用导致的顺序依赖和大量重试周期而阻塞。分析表明，尽管智能体展现了一些涌现式协调，但更清晰地划分角色可以降低总运行时和令牌成本。这揭示了边界追踪能够独特地捕获跨进程边界的应用层监控无法观察到的多智能体系统动态。

6 结论

本文介绍了 AgentSight，以弥合 AI 智能体的意图与其系统级

行为之间关键的语义鸿沟，采用了新颖的边界追踪方法。通过利用 eBPF，该系统无需任何插桩即可监控网络和内核事件，并通过混合关联引擎将 LLM 通信与其系统级影响进行因果关联。我们的评估表明，AgentSight 能够以低于 3% 的性能开销有效检测提示词注入攻击、推理循环和多智能体瓶颈。这种“用 AI 来监测 AI”的方式为日益自治的 AI 系统的安全可靠部署提供了基础方法论。

参考文献

[1] 2024. GPT-4o Mini: 推进高性价比智能体。<https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. 宣布GPT-4o Mini的博客文章。[2] 2025. crewAI: 编排角色扮演自主AI智能体的框架。<https://github.com/crewAIInc/crewAI>. GitHub仓库，访问日期：2025年7月30日。[3] 2025. 子智能体。<https://docs.anthropic.com/en/docs/claude-code/sub-agents>. 描述自定义AI子智能体的Claude Code文档页面。[4] Anthropic. 2025. 发布 Claude Code. <https://www.anthropic.com/news/claude-code>. 智能体编码工具发布公告，Anthropic博客。[5] Anysphere公司. 2025. Cursor: AI驱动的代码编辑器. <https://cursor.com/>. 具备智能体模式的AI辅助IDE；最新版本1.0于2025年6月4日发布。[6] Falco作者. 2023. Falco: 云原生运行时安全. <https://falco.org/> [7] Alexandr Bandurchin. [未注明日期.]. AI智能体可观测性详解：核心概念与标准。<https://uptrace.dev/blog/ai-agent-observability>. Uptrace博客，2025年4月16日。[8] Harrison Chase. 2023. LangChain: 通过可组合性使用大语言模型构建应用. <https://github.com/langchain-ai/langchain> [9] Linux 内核社区. 2023. BPF文档 - Linux内核。<https://www.kernel.org/doc/html/latest/bpf/> [10] Datadog公司. [未注明日期.]. 使用Datadog监控、排查和优化AI智能体。<https://www.datadoghq.com/blog/monitor-ai-agents/>. Datadog博客，2023年。[11] 董黎明、卢清华和朱黎明. 2024. AgentOps: 实现大语言模型智能体的可观测性. arXiv预印本 arXiv:2411.05285 (2024)。[12] eBPF社区. 2023. eBPF文档. <https://ebpf.io/> [13] eunomia-bpf社区. 2024. eBPF开发者教程。<https://github.com/eunomia-bpf/bpf-developer-tutorial>. 访问日期：2025-01-29。[14] Brendan Gregg. 2019. BPF性能工具。Addison-Wesley Professional。[15] 郭泰成、陈秀英、王亚琦、常瑞迪、裴世超、Nitesh V. Chawla、Olaf Wiest和张祥亮. 2024. 基于大语言模型的多智能体：进展与挑战综述. arXiv预印本 arXiv:2402.01680 (2024)。[16] Helicone. 2023. Helicone / 面向开发者的LLM可观测性。<https://www.helicone.ai/> [17] Been Kim、John Hewitt、Neel Nanda、Noah Fiedel和Oyvind Tafjord. 2025. 既然我们拥有了大语言模型，我们就能且应当追求智能体可解释性. arXiv预印本 arXiv:2506.12152 (2025)。[18] LangChain. 2023. 可观测性快速入门 - LangSmith - LangChain. <https://docs.smith.langchain.com/observability> [19] Langfuse. 2024. Langfuse - LLM可观测性与应用追踪. <https://langfuse.com/> [20] 刘光亚和Sujay Solomon. [未注明日期.]. AI智能体可观测性：不断演进的标准与最佳实践. <https://opentelemetry.io/blog/2025/ai-agent-observability/>. OpenTelemetry博客，2025年3月6日。

[21] Jannik Maierhöfer. 2025. AI Agent Observability with Langfuse. <https://langfuse.com/blog/2024-07-ai-agent-observability-with-langfuse>. Langfuse Blog, March 16, 2025.

[22] microsoft. [n. d.]. eBPF for Windows. <https://github.com/microsoft/ebpf-for-windows>.

[23] Dany Moshkovich and Sergey Zeltyn. 2025. Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems. *arXiv preprint arXiv:2507.11277* (2025).

[24] Taylor Mullen and Ryan J. Salva. 2025. Gemini CLI: Your Open Source AI Agent. <https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/>. Google Developers Blog, Jun 2025.

[25] Cilium Project. 2023. eBPF-based Networking, Observability, and Security. <https://cilium.io/>

[26] Benjamin Rombaut, Sogol Masoumzadeh, Kirill Vasilevski, Dayi Lin, and Ahmed E. Hassan. 2025. Watson: A Cognitive Observability Framework for the Reasoning of LLM-Powered Agents. *arXiv preprint arXiv:2411.03455* (2025).

[27] Aqua Security. 2023. Tracee: Runtime Security and Forensics using eBPF. <https://github.com/aquasecurity/tracee>

[28] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D. Nguyen. 2025. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. *arXiv preprint arXiv:2501.06322*

(2025).

[29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enable Next-Gen Large Language Model Applications. <https://github.com/microsoft/autogen>

[30] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. In-jecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics (ACL Findings)*. doi:10.48550/arXiv.2403.02691 arXiv:2403.02691.

[31] Boyang Zhang, Yicong Tan, Yun Shen, Ahmed Salem, Michael Backes, Savvas Zannettou, and Yang Zhang. 2024. Breaking Agents: Compromising Autonomous LLM Agents Through Malfunction Amplification. *arXiv preprint arXiv:2407.20859* (2024). Introduces an "infinite loop" attack that forces agents to repeat actions until max iteration..

[32] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 557–574.

[21] 扬尼克·迈尔霍费尔. 2025. 使用Langfuse实现AI智能体可观测性. <https://langfuse.com/blog/2024-07-ai-agent-observability-with-langfuse>. Langfuse博客, 2025年3月16日.[22] 微软. [n. d.]. eBPF for Windows. <https://github.com/microsoft/ebpf-for-windows>. [23] 达尼·莫什科维奇 和 谢尔盖·泽尔廷. 2025. 通过自动化驾驭不确定性: 观察、分析并优化智能体AI系统. arXiv预印本 arXiv:2507.11277 (2025). [24] 泰勒·穆伦 和 瑞安·J·萨尔瓦. 2025. Gemini CLI: 您的开源AI智能体. <https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/>. 谷歌开发者博客, 2025年6月.[25] Cilium项目. 2023. 基于eBPF的网络、可观测性和安全. <https://cilium.io/>[26] 本杰明·隆博, 索戈尔·马苏姆扎德, 基里尔·瓦西列夫斯基, 林大一, 以及 艾哈迈德·E·哈桑. 2025. Watson: 面向LLM驱动的代理推理的认知可观测性框架. arXiv预印本 arXiv:2411.03455 (2025).[27] Aqua安全. 2023. Tracee: 使用eBPF进行运行时安全和取证. <https://github.com/aquasecurity/tracee> [28] 阮青东, 道栋, 阮明东, 范国越, 巴里·奥沙利文, 以及 阮皇D. 2025. 多智能体协作机制: 大语言模型综述. arXiv预印本 arXiv:2501.06322

(2025年)。

[29] 吴青云, 加甘·班萨尔, 张洁宇, 吴怡然, 张少坤, 朱二康, 李贝宾, 蒋力, 张肖云, 以及王驰。2023. AutoGen: 赋能下一代大语言模型应用。 <https://github.com/microsoft/autogen> [30] 詹秋思, 梁志翔, 英子帆, 以及丹尼尔·康。2024. In-jecAgent: 工具集成大语言模型智能体中间接提示注入的基准测试。收录于《计算语言学协会发现（ACL发现）》。doi:10.48550/arXiv.2403.02691 arXiv:2403.02691。

[31] 张博洋, 谭一聪, 沈云, 艾哈迈德·萨利姆, 迈克尔·贝克斯, 萨瓦斯·赞内图, 以及张阳。2024. 破坏智能体: 通过故障放大危及自主LLM智能体。arXiv预印本arXiv:2407.20859（2024）。提出一种“无限循环”攻击, 迫使智能体重复操作直至达到最大迭代次数。[32] 郑宇声, 于通, YiweiYang, 胡艳鹏, 赖晓正, 丹·威廉姆斯, 以及安迪·奎恩。2025. 安全且高效地扩展应用。收录于《第19届USENIX操作系统设计与实现研讨会（OSDI 25）》。第557–574页。